

AI Desktop for macOS

MVP Architecture, Technology Stack, Product Scope & Development Roadmap

A system-wide AI assistant activated from anywhere on a Mac with a global keyboard shortcut.

Recommended MVP: Electron + React + TypeScript + Node.js/Fastify + PostgreSQL/Supabase + OpenAI API

Technical product blueprint

1. Product Vision

The product is a lightweight AI layer for macOS that is available from anywhere. The user presses a configurable global keyboard shortcut, a floating AI panel appears, and the user can immediately talk to an LLM. The key differentiator is automatic context: starting with selected text and later expanding to screen, application, browser, clipboard, and accessibility context.

```
Current app: Chrome / Slack / VS Code / Mail / Figma / etc.
|
Select text
|
Cmd + Shift + Space
|
Floating AI panel
|
Selected text = context
|
LLM
|
Streaming response
```

2. MVP Goal

Prove one core loop: from any macOS application, press a global shortcut, instantly open an AI panel, optionally carry selected text into the conversation, send the prompt to an LLM, stream the answer, and dismiss the panel with Esc.

MVP success criteria

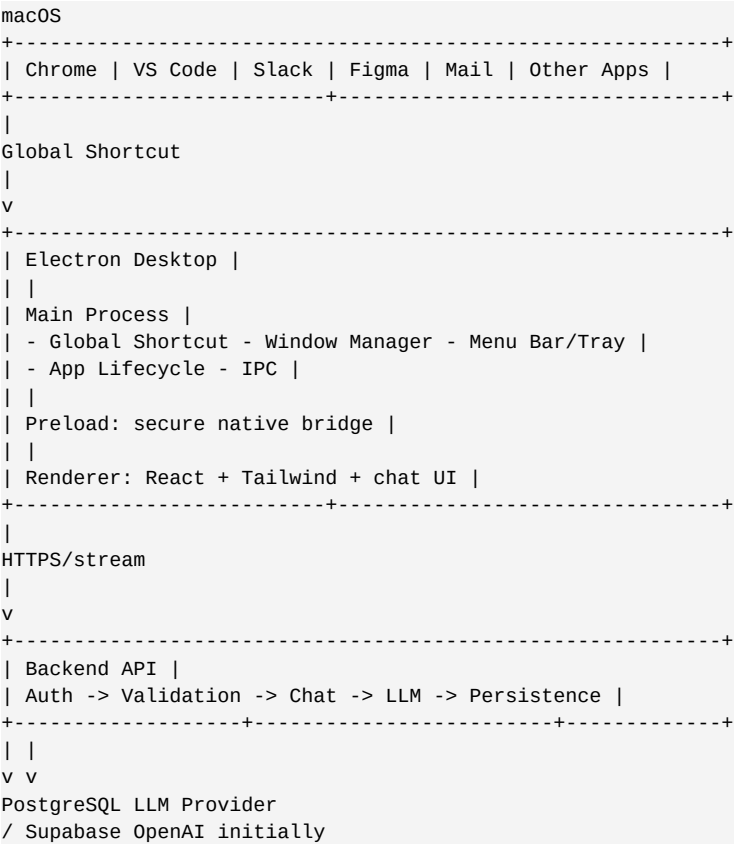
- Runs quietly in the macOS menu bar/background.
- Global shortcut works while another application has focus.
- AI panel opens quickly and focuses its input.
- Normal chat works without context.
- Selected text can be captured and presented as context.
- LLM responses stream.
- Conversations are persisted.
- LLM provider credentials never ship in the desktop app.
- The app can be packaged for macOS distribution.

3. MVP Feature Set

Feature	MVP	Notes
Global keyboard shortcut	Yes	Configurable; example Cmd + Shift + Space
Floating AI window	Yes	Frameless utility-style panel
Chat	Yes	Text input + user/assistant messages
Streaming	Yes	SSE or HTTP streaming
Selected text context	Yes	Clipboard-based first implementation
Conversation history	Yes	PostgreSQL persistence
Authentication	Yes	Supabase Auth or equivalent

Feature	MVP	Notes
Menu bar/tray	Yes	Open chat, settings, quit
Launch at login	Recommended	Makes assistant always available
Shortcut customization	Recommended	Avoid shortcut conflicts
Screen understanding	Later	ScreenCaptureKit + vision model
Accessibility context	Later	macOS Accessibility APIs
Actions/agents	Later	Not for first MVP

4. High-Level Architecture



5. Recommended Technology Stack

Layer	Technology	Reason
Desktop	Electron	Fast desktop shell and macOS integration
Language	TypeScript	Shared language across stack
UI	React	Fast iteration and mature ecosystem
Styling	Tailwind CSS	Rapid UI development
Build	Vite	Fast renderer builds
State	Zustand	Simple application state
Backend	Node.js + Fastify	Lightweight API layer
Validation	Zod	Runtime validation + TypeScript types

Layer	Technology	Reason
Database	PostgreSQL	Reliable relational storage
Auth	Supabase Auth	Fast MVP authentication
ORM	Drizzle	Type-safe and lightweight
AI	OpenAI API initially	Simple initial provider
Streaming	SSE / HTTP streaming	Responsive chat UX
Packaging	electron-builder	DMG/app packaging
Monorepo	pnpm + Turborepo	Shared types and clean workspace
Monitoring	Sentry	Recommended after prototype
Analytics	PostHog	Optional product analytics

6. Project Structure

```

ai-desktop/
|
+-- apps/
| +-- desktop/
| | +-- src/
| | +-- main/
| | | +-- index.ts
| | | +-- windows.ts
| | | +-- shortcuts.ts
| | | +-- tray.ts
| | | +-- ipc.ts
| | | +-- app-lifecycle.ts
| | +-- preload/
| | | +-- index.ts
| | | +-- api.ts
| | +-- renderer/
| | +-- components/
| | +-- pages/
| | +-- hooks/
| | +-- stores/
| | +-- lib/
| | +-- App.tsx
| | +-- main.tsx
| | +-- electron-builder.yml
| | +-- package.json
| | +-- vite.config.ts
| |
| +-- api/
| +-- src/
| +-- server.ts
| +-- routes/
| | +-- auth.ts
| | +-- chat.ts
| | +-- conversations.ts
| | +-- users.ts
| +-- services/
| | +-- llm/
| | | +-- openai.ts
| | | +-- llm.service.ts
| | +-- chat.service.ts
| | +-- conversation.service.ts
| +-- middleware/
| | +-- auth.ts
| | +-- rate-limit.ts
| +-- db/
| +-- client.ts
| +-- schema.ts
| +-- migrations/

```

```

|
+-- packages/
| +-- shared/
| | +-- types/
| | +-- schemas/
| | +-- constants/
| +-- config/
|
+-- package.json
+-- pnpm-workspace.yaml
+-- turbo.json
+-- .env.example

```

7. Electron Architecture

Main process

Own native desktop behavior: global shortcuts, window creation/positioning, tray/menu bar, app lifecycle and IPC.

```

main/
index.ts -> application entry
windows.ts -> AI window lifecycle
shortcuts.ts -> global shortcut registration
tray.ts -> menu bar icon/menu
ipc.ts -> IPC handlers
app-lifecycle.ts
config.ts

```

Preload

Expose a small explicit API to React. Keep context isolation enabled and Node integration disabled in the renderer.

```

window.desktop = {
  hideWindow(),
  showWindow(),
  getAppInfo(),
  onShortcut()
}

```

Renderer

React handles the UI: chat, messages, context preview, settings and local UI state.

8. Global Shortcut Flow

```

User is in Chrome / VS Code / Slack
|
Cmd + Shift + Space
|
v
Electron Main
|
globalShortcut
|
v
gather context
|
v
show AI window
|
v
focus chat input

```

The shortcut should be configurable. Registration can fail if another app already owns the combination, so the settings UI should support changing it.

9. Selected Text Context

The shortcut does not automatically provide selected text from every application. For the first implementation, use a clipboard-based capture strategy: obtain the selection, read it, and restore the user's previous clipboard content as quickly as practical.

```
Select text
|
Shortcut
|
Temporary copy / selection capture
|
Read selected text
|
AIContext.selectedText
|
Chat request
|
LLM
```

A robust later implementation can use macOS Accessibility APIs. Application behavior and permissions vary, so test Chrome, Safari, VS Code, Slack, Mail and common editors during the MVP.

10. Context Engine

Create a generic context layer even though the MVP only needs selected text. This keeps the chat system independent from context collection.

```
interface AIContext {
  selectedText?: string;
  clipboardText?: string;
  activeApplication?: {
    name: string;
    bundleId: string;
  };
  currentUrl?: string;
  screenshot?: string;
  accessibilityData?: unknown;
}

interface ContextProvider {
  getContext(): Promise<AIContext>;
}
```

Future providers: SelectedTextProvider, ClipboardProvider, ActiveAppProvider, ScreenProvider, BrowserProvider and AccessibilityProvider.

11. Chat Request Lifecycle

```
React ChatInput
|
Desktop API client
|
POST /v1/chat
|
Backend auth middleware
|
Zod validation
|
Chat Service
|
LLM Service
|
OpenAI Provider
|
streaming response
|
```

```
Backend -> Electron -> React
|
Save completed messages
```

12. API Design

Method	Endpoint	Purpose
POST	/v1/chat	Send message + optional context and stream response
POST	/v1/conversations	Create conversation
GET	/v1/conversations	List user's conversations
GET	/v1/conversations/:id	Retrieve conversation
DELETE	/v1/conversations/:id	Delete conversation
GET	/v1/users/me	Return authenticated profile

```
POST /v1/chat
{
  "conversationId": "abc123",
  "message": "Explain this simply",
  "context": {
    "type": "selected_text",
    "text": "The selected text..."
  }
}
```

13. LLM Provider Abstraction

Keep provider-specific SDK calls behind a common interface so other model providers can be added later.

```
interface LLMPProvider {
  streamChat(
    messages: Message[],
    options?: LLMOptions
  ): AsyncIterable<string>;
}

LLMService
-> OpenAIProvider
-> AnthropicProvider (future)
-> GeminiProvider (future)
```

14. Database Schema

```
users
-----
id
email
created_at

conversations
-----
id
user_id
title
created_at
updated_at

messages
-----
id
conversation_id
```

```
role
content
model
created_at
```

```
User -> many Conversations -> many Messages
```

Do not add a vector database, Redis, event bus or microservices unless a real MVP requirement demands them.

15. Authentication & Secrets

Use Supabase Auth or a similar hosted auth system. The desktop app receives a session/token and sends it to the backend. The backend validates identity before accessing data or calling the LLM.

```
Electron
|
JWT / session
v
Backend
|
+-- validate identity
+-- authorize user data
+-- call LLM
```

Never put the OpenAI API key in the Electron app. Keep provider credentials on the backend.

16. Electron Security

- contextIsolation: true
- nodeIntegration: false
- Use a narrow preload API
- Validate IPC payloads
- Do not expose arbitrary shell execution
- Use secure credential storage where appropriate
- Use HTTPS
- Apply rate limits and request size limits

17. macOS Permissions & Native Features

Capability	Technology	Phase
Global shortcut	Electron globalShortcut	MVP
Floating panel	Electron BrowserWindow / macOS panel behavior	MVP
Selected text	Clipboard first; Accessibility later	MVP/V2
Screen capture	ScreenCaptureKit	V2
App UI inspection	macOS Accessibility APIs	V2/V3
Browser context	Browser extension or browser-specific integration	Later

18. UI / UX Specification

```

+-----+
|  AI Settings|
+-----+
|  |
| Context (only when available) |
| +-----+ |
| | Selected text... | |
| +-----+ |
|  |
| Assistant response |
|  |
+-----+
| Ask anything...  |
+-----+

```

The panel should feel like a Mac utility rather than a traditional application: fast to open, automatically focused, dismissible with Esc, minimal chrome, and capable of preserving conversation state.

19. Menu Bar Experience

```

Menu bar icon
|
+-- Ask AI
+-- New conversation
+-- Settings
+-- Keyboard shortcut
+-- Account
+-- Quit

```

The menu bar is secondary. The keyboard shortcut is the primary interaction.

20. Packaging & Distribution

Use electron-builder to produce the macOS .app and .dmg. Public distribution should include Apple code signing and notarization. Add automatic updates after the core app is stable.

```

Electron build
|
.app / .dmg
|
Code signing
|
Notarization
|
Distribution

```

21. Deployment Architecture

```

Internet
|
v
Electron App
|
HTTPS
|
v
Node / Fastify API
/ v v
PostgreSQL LLM API
/ Supabase OpenAI

```

Use managed infrastructure for the MVP. There is no need for Kubernetes, complex AWS networking or a distributed microservice architecture.

22. Development Plan

Phase	Scope	Target
Prototype	Electron + React + shortcut + basic chat	1–3 days
MVP Core	Backend + LLM + streaming + persistence	Several days
MVP Polish	Auth + menu bar + settings + packaging	Several days
Context V1	Selected text + clipboard + active app	After core
Context V2	Screen capture + multimodal AI	Later
Agent V1	Actions and integrations	Later

23. Recommended Build Order

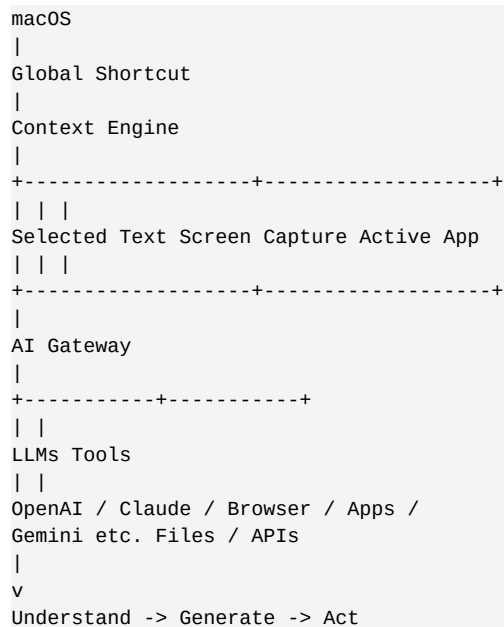
- Create Electron + React + TypeScript project.
- Create hidden/secondary AI BrowserWindow.
- Register global keyboard shortcut.
- Show/focus AI window when shortcut fires.
- Add Esc-to-hide.
- Build minimal chat UI.
- Create Fastify backend.
- Add authentication.
- Create PostgreSQL schema.
- Implement LLM provider service.
- Implement streaming chat.
- Persist conversations/messages.
- Add selected-text context.
- Add menu bar/tray.
- Add shortcut customization.
- Add launch-at-login.
- Package and test the macOS app.

24. What NOT to Build in MVP

- Multi-agent systems
- RAG/vector database
- Voice assistant
- Complex memory system
- Computer-use automation
- Browser automation
- Dozens of app integrations
- Windows/Linux support

- Enterprise administration
- Team collaboration
- Multiple LLM vendors
- Kubernetes/microservice infrastructure

25. Future Architecture



26. Product Evolution

Version	Experience
V0.1	Global shortcut → floating chat → LLM
V0.2	Selected text → automatic AI context
V0.3	Clipboard + active application context
V0.4	Screen understanding with vision models
V1	Context-aware desktop assistant
V2	Approved actions, integrations and agents

27. Difficulty Assessment

Area	Difficulty	Comment
Electron shell	Low	Desktop application foundation
Global shortcut	Low–Medium	Straightforward; handle conflicts
Floating panel	Low–Medium	Needs polished macOS behavior
LLM integration	Low	Standard API work
Streaming	Low–Medium	Clean client/server stream handling

Area	Difficulty	Comment
Auth + database	Medium	Standard web engineering
Selected text	Medium	Cross-app behavior and permissions
Screen understanding	Medium–High	Capture permissions + vision processing
Accessibility automation	High	More complex macOS integration
Agent actions	High	Safety, permissions and reliability

28. Final Recommendation

Build the first production-capable MVP as a single Electron desktop application plus a single backend API. The desktop app owns the experience and macOS integration; the backend owns authentication, conversations, usage controls and LLM calls. Keep a Context Engine and LLM Provider abstraction from day one, but keep their first implementations extremely small.

Core product principle: build the interaction before the infrastructure. The MVP wins if it feels instant: shortcut → context → AI → answer.